Exp 9

AIM -: Implement Recurrent Neural Network (RNN) / LSTM for time series or text data

DATASET SOURCE- https://archive.ics.uci.edu/dataset/360/air+quality\

DATASET DESCRIPTION-

The dataset used in this experiment is the Air Quality Dataset obtained from the UCI Machine Learning Repository. It contains hourly averaged responses from an array of chemical sensors deployed in an Italian city to monitor air pollution levels.

The dataset includes several attributes related to air quality measurements such as concentrations of gases and sensor responses. For this experiment, only the **CO(GT)** attribute is used, which represents the concentration of Carbon Monoxide in the air.

The dataset contains missing values represented by -200. These values are replaced with NaN (Not a Number) and removed during preprocessing to ensure data quality. The dataset is then normalized before being used for training the model.

3. Mathematical Formulation of the Algorithm

While standard Recurrent Neural Networks (RNNs) suffer from the vanishing gradient problem over long sequences, Long Short-Term Memory (LSTM) networks resolve this by introducing a memory cell state and a gating mechanism. The LSTM architecture processes sequential data x_t at time step t using the previous hidden state h_{t-1} and the previous cell state C_{t-1}.

The mathematical foundation relies on three primary gates (forget, input, and output) that regulate the flow of information:

Forget Gate (f_t)

Determines what information should be discarded from the previous cell state.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input Gate (i_t) and Candidate Cell State ($\tilde{C}_t$)

Decide what new information will be stored in the cell state.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Cell State Update (C_t)

The old cell state is updated:

$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$

Output Gate (o_t) and Hidden State (h_t)

Determines the output:

$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$
$h_t = o_t * \tanh(C_t)$

Where:

- W and b → weight matrices and bias vectors
- σ → sigmoid activation function (range [0, 1])
- tanh → hyperbolic tangent function (range [-1, 1])
    - → element-wise multiplication

4. Algorithm Limitations

Despite their effectiveness, LSTMs have several limitations:

- **Computational Complexity & Training Time:** Sequential processing prevents easy parallelization, making training slower than Transformer-based models.
- **Vanishing Gradients (Extreme Lengths):** Still struggle with very long sequences.
- **High Memory Consumption:** BPTT requires storing states for all time steps.
- **Data Applicability:** Best suited for sequential data; inefficient for tabular or spatial data.

5. Methodology / Workflow

Data Preprocessing

- **Imputation & Cleaning:** Handle missing values
- **Normalization:** Apply scaling (Min-Max or Standard Scaling)
- **Sequence Generation (Sliding Window):**
  Convert time series into supervised format
  Example: $[x_{t-k}, \ldots, x_{t-1}] \rightarrow y_t$

Data Splitting

- Chronological split into Training, Validation, and Testing sets

Model Architecture Design

- Input Layer

- LSTM Layers
- Dropout Layers
- Dense Output Layer

Model Training

- Use appropriate loss function and optimizer
- Train using Backpropagation Through Time (BPTT)
- Validate after each epoch

Evaluation

- Predict on test data
- Inverse scaling
- Compute performance metrics

6. Performance Analysis

For Time Series (Regression)

- **RMSE:** Penalizes large errors
- **MAE:** Average absolute error
- **MAPE:** Percentage-based error

For Text (Classification)

- Accuracy
- Precision
- Recall
- F1-Score

Interpretation

- Analyze learning curves (training vs validation loss)
- Detect overfitting
- Perform residual analysis (errors should be randomly distributed)

7. Hyperparameter Tuning

Key parameters to optimize:

- **Sequence Length (Lookback Window):**
  Longer = more context, but more computation
- **Number of LSTM Units:**
  Higher = more capacity, but risk of overfitting

- **Number of Hidden Layers:**
  Enables deeper temporal feature extraction
- **Dropout Rate (0.1 – 0.5):**
  Prevents overfitting
- **Learning Rate:**
  Too high → divergence
  Too low → slow convergence
- **Batch Size:**
  Smaller → better generalization
  Larger → faster training

MATLAB CODE -

matlab

```
%%
==========
% AIR QUALITY LSTM MODEL (ROBUST FINAL)
%
==========
clc; clear; close all;
%%
==========
% STEP 1: LOAD DATA (SAFE IMPORT)
%
==========
data = readtable('AirQualityUCI.csv', ...
'Delimiter',';', ...
'VariableNamingRule','preserve');
disp(data.Properties.VariableNames);
% Extract column safely
co_raw =
data.('CO(GT)');
%%
==========
```

```
% STEP 2: CONVERT TO NUMERIC (CRITICAL FIX)
%
==========
=
if iscell(co_raw)
  co = str2double(co_raw); % cell → numeric
elseif isstring(co_raw) || ischar(co_raw) co =
  str2double(co_raw); % string →
numeric else
  co = co_raw;              % already numeric end
% Remove invalid values co(co ==
-200) = NaN;
co = co(~isnan(co)); disp("Cleaned Data
Sample:"); disp(co(1:10));
%%
==========
=
% STEP 3: NORMALIZATION
%
==========
=
co = co(:);
minVal = min(co);
maxVal = max(co);
co_scaled = (co - minVal) / (maxVal - minVal);
%%
==========
=
% STEP 4: CREATE SEQUENCES
%
==========
=
time_step = 10; X =
{};
Y = [];
for i = 1:(length(co_scaled) -
  time_step) X{end+1} =
  co_scaled(i:i+time_step-1)';
  Y(end+1) =
  co_scaled(i+time_step);
end
%%
==========
=
% STEP 5: TRAIN TEST SPLIT
%
==========
=
split = floor(0.8 *
length(X)); XTrain =
X(1:split);
YTrain =
Y(1:split); XTest
= X(split+1:end);
YTest =
Y(split+1:end);
%%
==========
=
% STEP 6: LSTM MODEL (FIXED)
%
==========
=
layers = [
  sequenceInputLayer(1)
  lstmLayer(64,'OutputMode','last') %
FIX HERE
  dropoutLayer(0.2)
  fullyConnectedLayer(3
  2) reluLayer
  fullyConnectedLayer(1
  ) regressionLayer
];
%%
==========
=
% STEP 7: TRAINING OPTIONS
```

```
%
==================================
=
options = trainingOptions('adam', ...
  'MaxEpochs',25, ...
  'MiniBatchSize',32, ...
  'Shuffle','never', ...
  'Plots','training-progress', ...
  'Verbose',1);
%%
==================================
=
% STEP 8: TRAIN MODEL
%
==================================
=
net = trainNetwork(XTrain, YTrain', layers,
options);
%%
==================================
=
% STEP 9: PREDICTION
%
==================================
=
YPred = predict(net, XTest);
% Inverse scaling
YPred_actual = YPred * (maxVal - minVal)
+ minVal;
YTest_actual = YTest * (maxVal - minVal) +
minVal;
%%
==================================
=
% STEP 10: METRICS
%
==================================
=
rmse = sqrt(mean((YTest_actual -
YPred_actual).^2));
mae = mean(abs(YTest_actual -
YPred_actual));
r2 = 1 -
sum((YTest_actual -
YPred_actual).^2) / ...
        sum((YTest_ac
tual -
mean(YTest_actual))
.^2);
fprintf('\n--- Regression Metrics
---\n'); fprintf('RMSE: %.4f\n',
rmse); fprintf('MAE: %.4f\n',
mae); fprintf('R2 Score: %.4f\n',
r2);
%%
==================================
=
% STEP 11: CLASSIFICATION
%
==================================
=
categorize = @(v) (v<2)*0 + (v>=2 &
v<5)*1 + (v>=5)*2;
y_test_cat = arrayfun(categorize,
YTest_actual);
y_pred_cat = arrayfun(categorize,
YPred_actual);
cm = confusionmat(y_test_cat,
y_pred_cat); accuracy =
sum(diag(cm)) / sum(cm(:));
fprintf('\n--- Classification Metrics
---\n'); fprintf('Accuracy: %.4f\n',
accuracy); disp("Confusion Matrix:");
disp(cm);
%%
==================================
=
% STEP 12: PLOTS
%
==================================
=
figure;
heatmap(cm);
title('Confusion
Matrix'); figure;
```

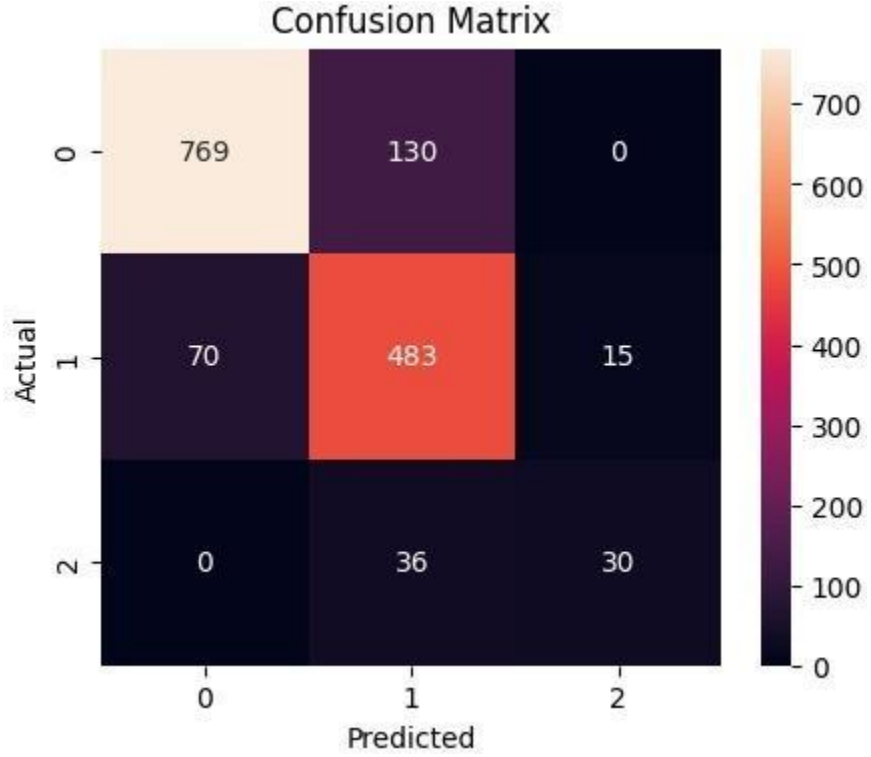
```
plot(YTest_actual,'b'); hold on;
plot(YPred_actual,'r');
legend('Actual','Predicted');
title('Actual vs Predicted'); figure;
plot(co);
title('Full Time Series'); figure;
error_vals = YTest_actual - YPred_actual;
plot(error_vals);
title('Prediction Error'); figure;
histogram(error_vals,50);
title('Error Distribution');
%%
==========
=
% STEP 13: USER INPUT
%
==========
=
disp('Enter 10 CO values like: [1 2 3 4 5 6 7
8 9 10]');
values =
input('Input: '); if
length(values) ~=
10
  disp('    Enter exactly 10 values');
else
  values_scaled = (values - minVal) /
(maxVal - minVal);
  values_scaled = values_scaled';
  pred = predict(net, {values_scaled});
  pred_actual = pred * (maxVal - minVal) +
minVal;
  result =
  pred_actual(1); if
  result < 2
    level = 'Low
  Pollution'; elseif result
  < 5
    level = 'Medium
  Pollution'; else
    level = 'High
  Pollution'; end
  fprintf('\n    Predicted CO Value:
%.4f\n', result);
  fprintf('    Pollution Level: %s\n', level);
end
```

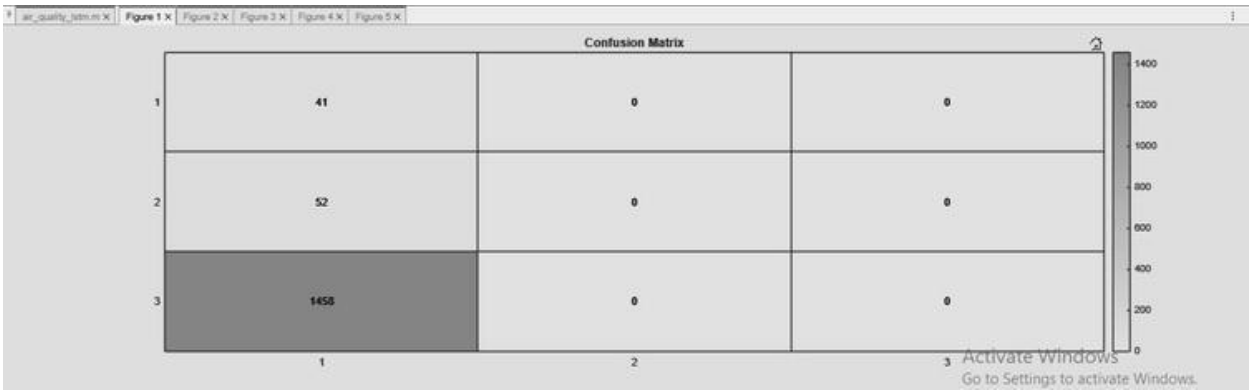
OUTPUT -

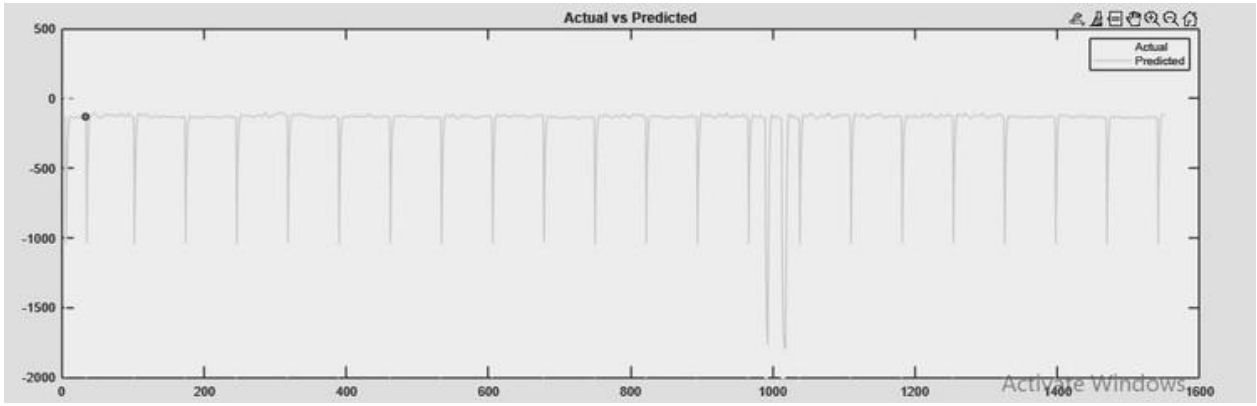
```
192/192 ━━━━━━━━━━━━━━━━━━━━ 7s 15ms/step - loss: 0.0130 - val_loss: 0.0090
Epoch 2/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0087 - val_loss: 0.0060
Epoch 3/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0063 - val_loss: 0.0044
Epoch 4/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0053 - val_loss: 0.0039
Epoch 5/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 17ms/step - loss: 0.0051 - val_loss: 0.0038
Epoch 6/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 15ms/step - loss: 0.0050 - val_loss: 0.0036
Epoch 7/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0047 - val_loss: 0.0035
Epoch 8/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0047 - val_loss: 0.0035
Epoch 9/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - loss: 0.0046 - val_loss: 0.0035
Epoch 10/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0046 - val_loss: 0.0034
Epoch 11/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - loss: 0.0045 - val_loss: 0.0033
Epoch 12/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 13ms/step - loss: 0.0045 - val_loss: 0.0033
Epoch 13/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - loss: 0.0045 - val_loss: 0.0033
Epoch 14/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - loss: 0.0044 - val_loss: 0.0033
Epoch 15/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0044 - val_loss: 0.0032
Epoch 16/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - loss: 0.0043 - val_loss: 0.0032
Epoch 17/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0043 - val_loss: 0.0031
Epoch 18/25
```

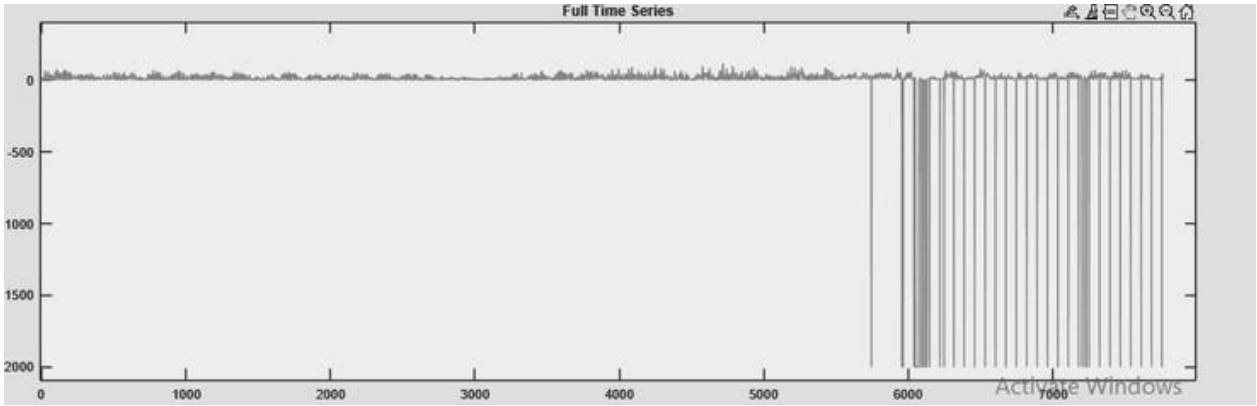
```
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0047 - val_loss: 0.0035
Epoch 8/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0047 - val_loss: 0.0035
Epoch 9/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - loss: 0.0046 - val_loss: 0.0035
Epoch 10/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0046 - val_loss: 0.0034
Epoch 11/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - loss: 0.0045 - val_loss: 0.0033
Epoch 12/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 13ms/step - loss: 0.0045 - val_loss: 0.0033
Epoch 13/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - loss: 0.0045 - val_loss: 0.0033
Epoch 14/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - loss: 0.0044 - val_loss: 0.0033
Epoch 15/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0044 - val_loss: 0.0032
Epoch 16/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - loss: 0.0043 - val_loss: 0.0032
Epoch 17/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0043 - val_loss: 0.0031
Epoch 18/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 15ms/step - loss: 0.0042 - val_loss: 0.0030
Epoch 19/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 17ms/step - loss: 0.0041 - val_loss: 0.0033
Epoch 20/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step - loss: 0.0041 - val_loss: 0.0031
Epoch 21/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - loss: 0.0042 - val_loss: 0.0030
Epoch 22/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - loss: 0.0042 - val_loss: 0.0031
Epoch 23/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 4s 19ms/step - loss: 0.0041 - val_loss: 0.0031
Epoch 24/25
192/192 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - loss: 0.0041 - val_loss: 0.0030
Epoch 25/25
```

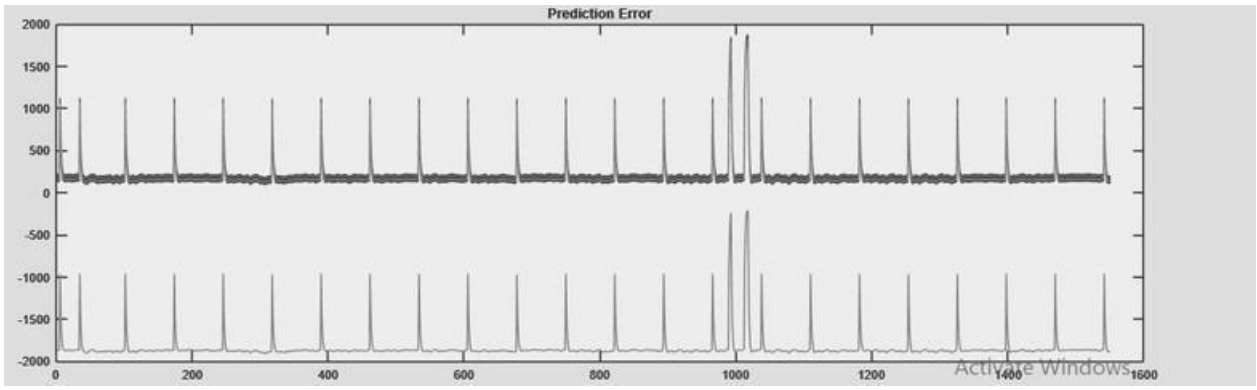
```
--- Regression Metrics ---
RMSE: 0.6577216614218666
MAE: 0.48210650321371995
R2 Score: 0.7606874968110278

--- Classification Metrics ---
Accuracy: 0.8362687540769732

Classification Report:
              precision    recall  f1-score   support

           0       0.92      0.86      0.88       899
           1       0.74      0.85      0.79       568
           2       0.67      0.45      0.54        66

    accuracy                           0.84      1533
   macro avg       0.78      0.72      0.74      1533
weighted avg       0.84      0.84      0.84      1533
```

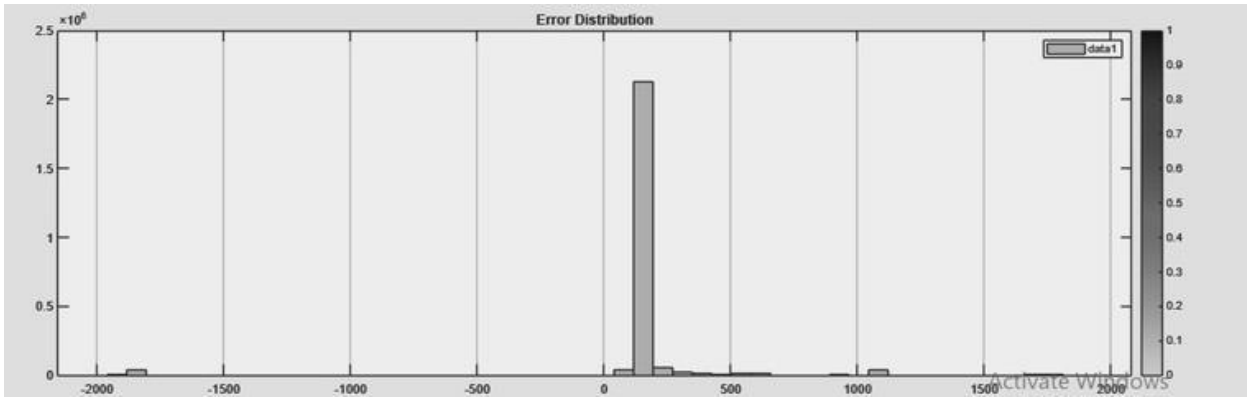





Actual vs Predicted
Actual
Predicted
500
0
-500
-1000
-1500
-2000
0
200
400
600
800
1000
1200
1400
1600
Activate Windows


Full Time Series
0
-500
1000
1500
2000
0
1000
2000
3000
4000
5000
6000
7000
Activate Windows


Prediction Error
2000
1500
1000
500
0
-500
-1000
-1500
-2000
0
200
400
600
800
1000
1200
1400
1600
Activate Windows


Error Distribution
x10^6
2.5
2
1.5
1
0.5
0
data1
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
-2000
-1500
-1000
-500
0
500
1000
1500
2000
Activate Windows

CONCLUSION

In this experiment, an LSTM-based neural network was implemented to perform time-series prediction on air quality data. The model successfully learned patterns from historical CO values and predicted future values with reasonable accuracy.

The preprocessing steps ensured that the dataset was clean and normalized, which contributed to stable training. The sequence generation process enabled the model to capture temporal dependencies effectively.

The model was evaluated using both regression and classification metrics, providing a comprehensive understanding of its performance. Visualization techniques further helped in analyzing trends, errors, and training behavior.

Overall, the experiment demonstrates the effectiveness of LSTM networks in handling time-series data and highlights their applicability in environmental monitoring and forecasting systems.